

Entrenamiento PROGRAMACIÓN COMPETITIVA

Desarrolla tu lógica de programación

- Terminología Básica
- Tiempo de Ejecución
- Programación Competitiva
- Introducción a C++
- Entrada/Salida
- Tipos de datos
- Condicionales
- Bucles o Ciclos



Terminología Básica

TEMARIO

1. Lenguaje Binario
2. Lenguaje de Programación
3. Compilación
4. Ejecución
5. Tiempo de Ejecución
6. Complejidad Algorítmica

Lenguaje Binario

Terminología Básica

- El lenguaje binario es el único lenguaje que las computadoras entienden.
- Está compuesto únicamente por **ceros (0)** y **unos (1)**.
- Las computadoras funcionan con electricidad, y la manera más eficiente de representarla es con dos estados:
 -  **1 (Encendido)**
 -  **0 (Apagado)**



Lenguaje de Programación

Terminología Básica

- Un lenguaje de programación es un conjunto de reglas que nos permiten darle instrucciones a una computadora **de forma comprensible para nosotros**.
- Escribir directamente en binario sería casi imposible para proyectos grandes.
- Los lenguajes de programación permiten escribir código de manera más clara y organizada.

```
package main

import "fmt"

func main() {
    fmt.Printf("hello, world")
}
```

Compilación

Proceso de traducción de un Lenguaje de Programación a Lenguaje Binario

```
0101010111101110001101
0100010100010101001010
0101010010101010000101
0011010001010100011110
0110010100101010101001
1110001101010010010001
```

Lenguaje binario

Ejecución

Terminología Básica

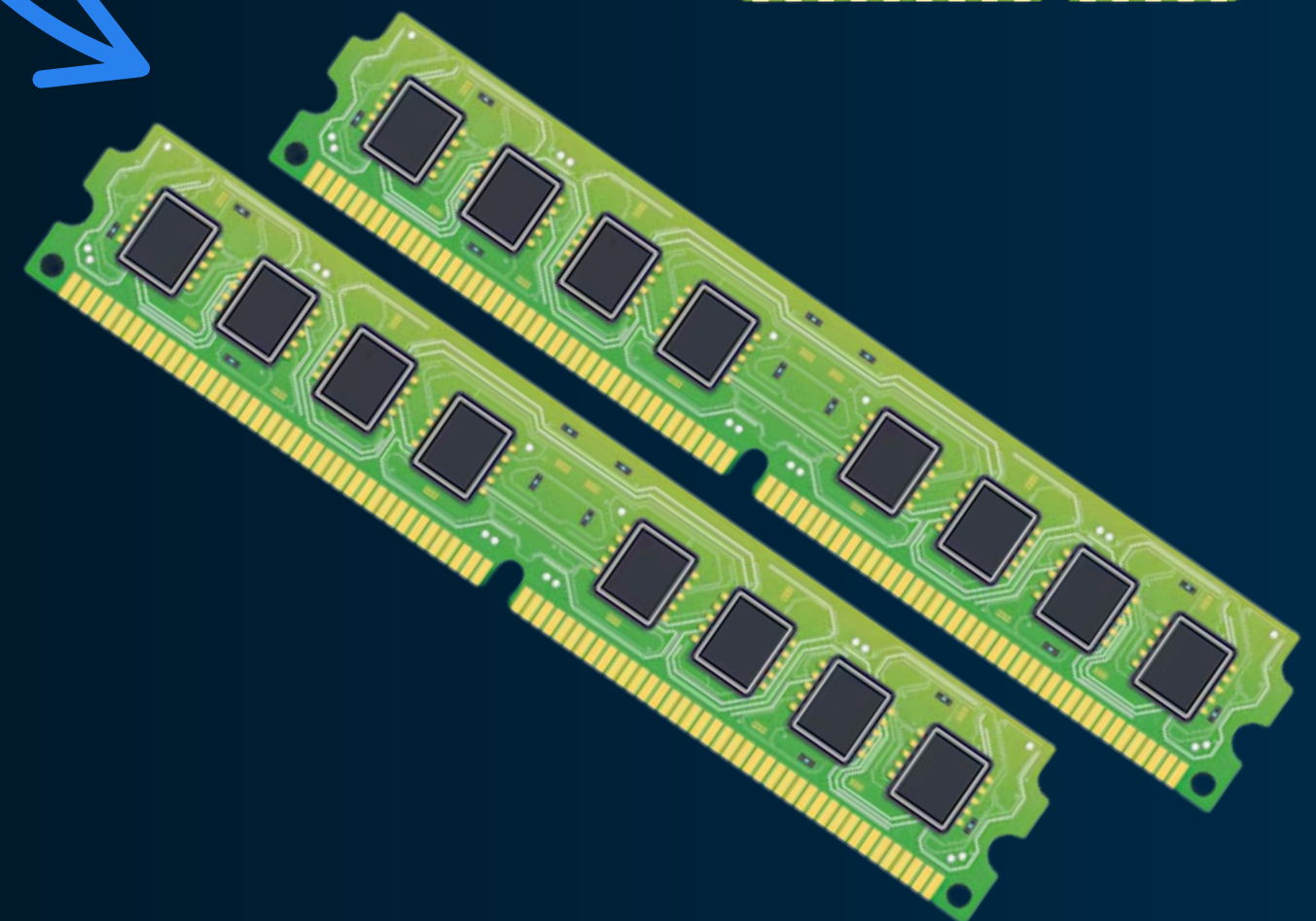
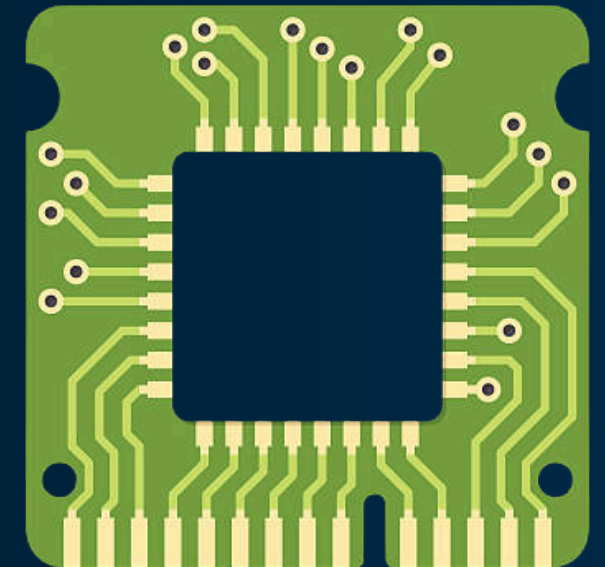
- Proceso por el cual la computadora ejecuta las instrucciones del código ya compilado.
- La ejecución de un programa puede ser **más lenta** o **más rápida** dependiendo de las instrucciones que llevará a cabo el programa.

TIEMPO DE EJECUCIÓN

- Se mide en función del tamaño de entrada "**N**".
- Se denota de la siguiente forma: **$O(N)$**

Lenguaje Compilado

```
0101010111101110001101
0100010100010101001010
0101010010101010000101
0011010001010100011110
0110010100101010101001
1110001101010010010001
```



Espacio de Preguntas

RESUMEN

- ✓ Las computadoras solo entienden binario (0s y 1s).
- ✓ Usamos lenguajes de programación porque escribir en binario es muy difícil.
- ✓ El compilador traduce nuestro código a binario para que la computadora lo entienda.
- ✓ El programa puede ejecutarse más rápido o más lento dependiendo de las indicaciones del programador.

Programación Competitiva

- La programación competitiva es una disciplina en la que los programadores **resuelven problemas** bajo restricciones de **tiempo** y **memoria**.
- Se trata de escribir código eficiente y correcto en el menor tiempo posible.
- Es un campo donde la **lógica**, la **optimización** y la **creatividad** son esenciales.



Introducción al Lenguaje

TEMARIO

1. Función Principal o Main
2. Tipos de Datos
3. Entrada y Salida de Datos
4. Operaciones con números
5. Condicionales
6. Bucles o Ciclos

Función Principal

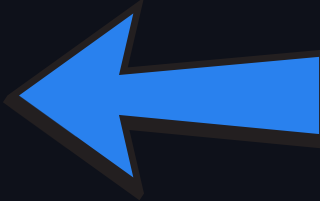
Introducción a C++

- En muchos lenguajes de programación es obligatorio usar siempre una función principal o **función main**.
- Es importante saber que dentro de esta función irá todo nuestro código principal.
- A diferencia de otros lenguajes como Java, en C++ es necesario colocar al final de esta función la línea: **return 0;**



```

1  int main() {
2
3      NUESTRO
4      CÓDIGO
5      IRÁ AQUÍ
6
7      return 0;
8  }
```




```

1  public class Main {
2      public static void main(String[] args) {
3
4          NUESTRO CÓDIGO
5          DE JAVA VA AQUÍ
6
7      }
8  }
```

Tipos de Datos

Introducción a C++

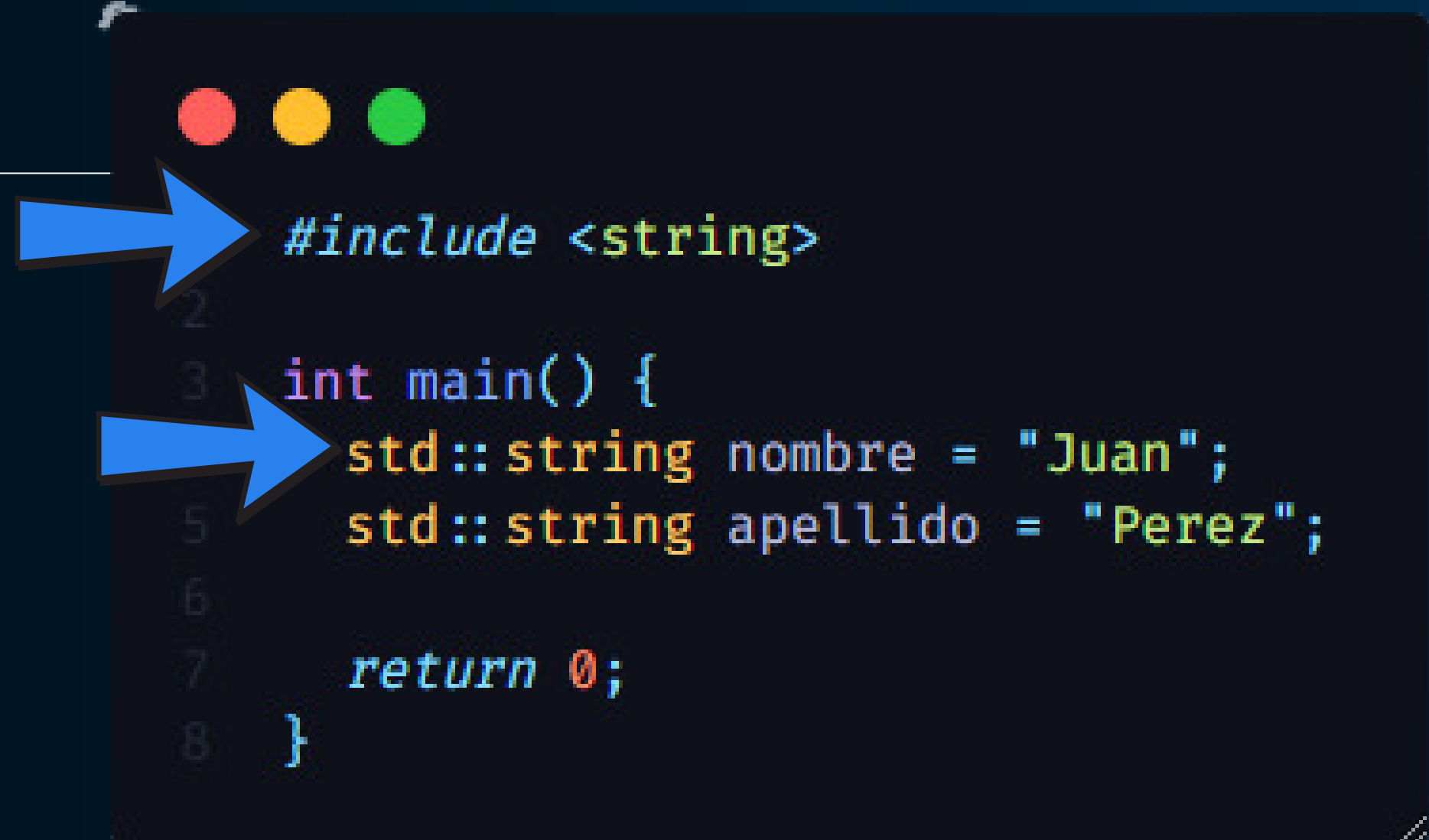
- En todo lenguaje de programación podemos **almacenar un valor en una variable**. Por ejemplo, en matemáticas podemos decir que una variable X tiene el valor de 10 ($x = 10$).
- En C++, nosotros **debemos especificar cuál es el tipo de dato** que almacenará una variable:
 - Tipos de datos **numéricos**
 - Tipos de datos **booleanos** (verdadero o falso)
 - Tipos de datos **de texto**

```
1  int main() {
2      int entero = 10;
3      float decimal = 3.14;
4      double decimalMasGrande = 3.1416;
5      long long enteroGrande = 1000000000000000000;
6
7      bool booleano = true;
8
9      char character = 'a';
10
11     return 0;
12 }
```

Cadena de texto

Introducción a C++

- A diferencia de los tipos de datos anteriores, las cadenas de texto **requieren de la inclusión de una librería llamada "string"**. Esto se hace con la línea de código: `#include <string>`
- Si no importamos la librería, entonces ocurrirá un error.



```
1 #include <string>
2
3 int main() {
4     std::string nombre = "Juan";
5     std::string apellido = "Perez";
6
7     return 0;
8 }
```

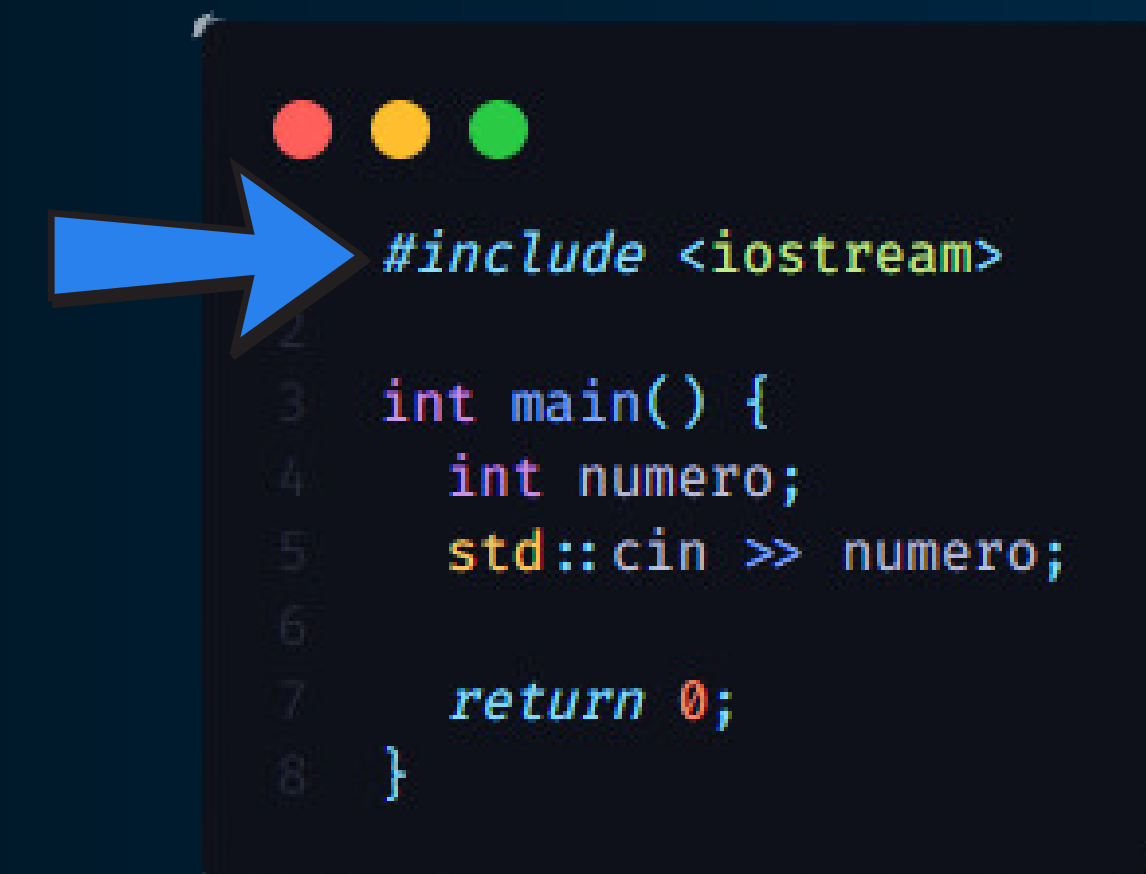
Entrada y Salida de Datos

Introducción a C++

- Un programa puede tener la capacidad de **recibir datos del usuario** (como cuando un formulario nos pide nuestro nombre y edad).
- También puede tener la capacidad de **enviar datos al usuario** (como cuando un programa de calculadora nos muestra la suma de dos números).
- En C++, la entrada y salida de datos se realiza usando la librería **<iostream>**. Si no la incluimos, ocurrirá un error.



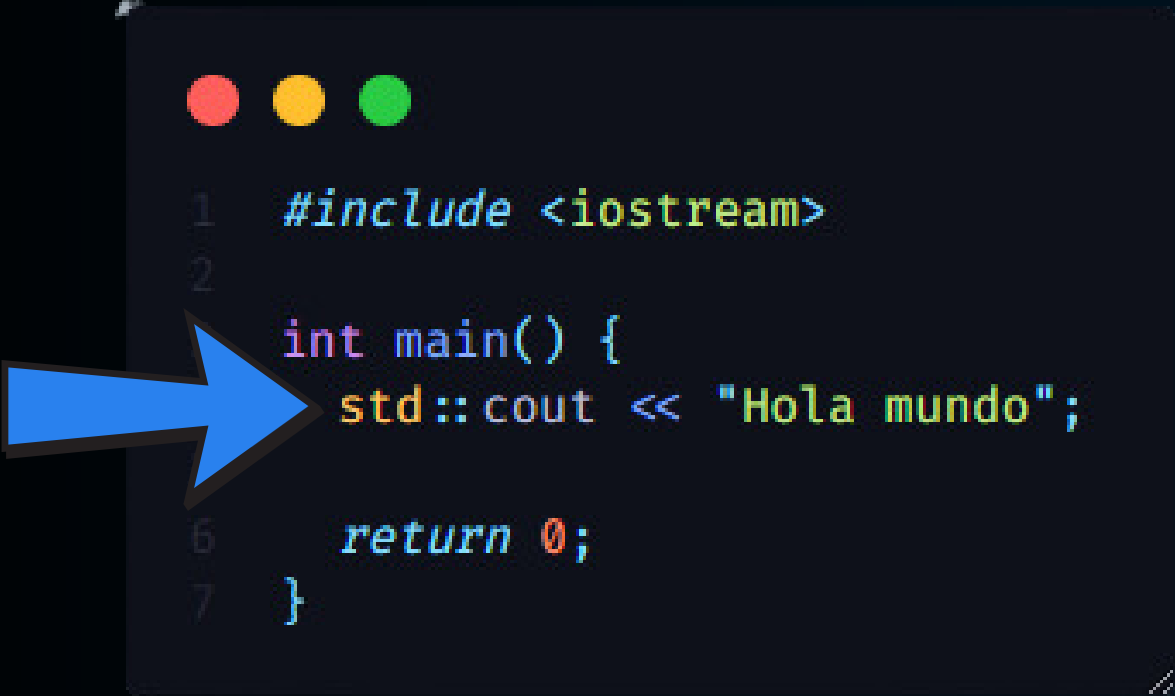
```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hola mundo";
5
6     return 0;
7 }
```



```
1 #include <iostream>
2
3 int main() {
4     int numero;
5     std::cin >> numero;
6
7     return 0;
8 }
```


Salida de Datos

- Para imprimir "Hola mundo", escribimos la siguiente línea de código: **std::cout << "Hola mundo";**
- **COUT** significa **Character Output** (salida de caracteres)

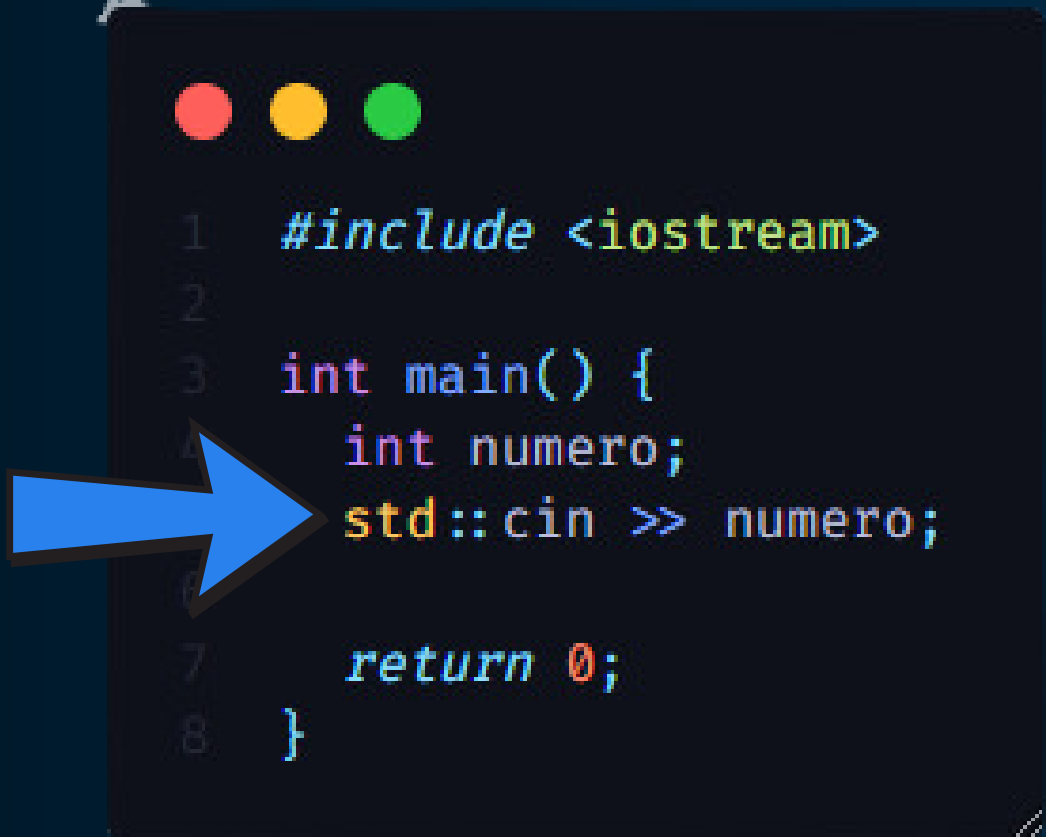


```

1  #include <iostream>
2
3  int main() {
4      std::cout << "Hola mundo";
5
6      return 0;
7  }
    
```

Entrada de Datos

- Para pedir que el usuario ingrese un número, escribimos la siguiente línea de código: **std::cin >> numero;**
- **numero** es un tipo de dato numérico entero (integer)
- **CIN** significa **Character Input** (entrada de caracteres)



```

1  #include <iostream>
2
3  int main() {
4      int numero;
5      std::cin >> numero;
6
7      return 0;
8  }
    
```

Escribe menos, haz lo mismo

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      std::string saludo = "Hola mundo";
6      std::cout << saludo;
7
8      return 0;
9  }
```

- Podemos evitar usar el prefijo **std::** en **std::cout** y **std::cin** usando la siguiente línea de código: **using namespace std;**

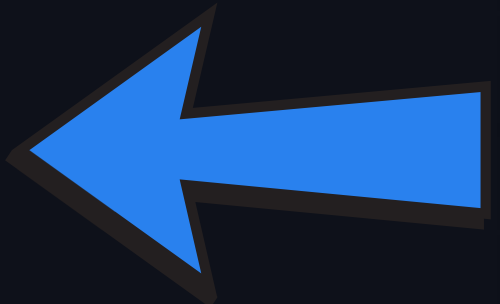
```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main() {
7      string saludo = "Hola mundo";
8      cout << saludo;
9
10     return 0;
11 }
```

Escribe menos, haz lo mismo

- Podemos **incluir todas las librerías de C++** en una sola línea de código.
- Esto es posible escribiendo la línea de código: **`#include <bits/stdc++.h>`**

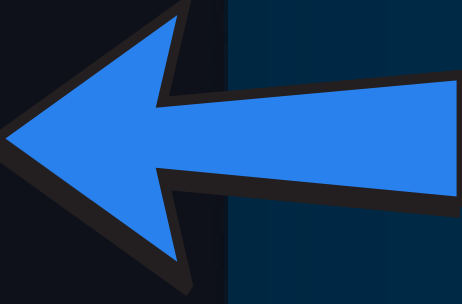
```

1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main() {
7      string saludo = "Hola mundo";
8      cout << saludo;
9
10     return 0;
11 }
```



```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      string saludo = "Hola mundo";
7      cout << saludo;
8
9      return 0;
10 }
```



Operaciones Matemáticas

Introducción a C++

- También podemos realizar operaciones matemáticas con tipos de datos numéricos.

Operación	Operador	Ejemplo	Acción
Suma	+	$X + Y$	Suma X más Y
Resta	-	$X - Y$	Resta X menos Y
Multiplicación	*	$X * Y$	Multiplica X por Y
División	/	X / Y	Divide X entre Y, regresa el cociente
Potencia	pow	pow(X,Y)	Eleva X a la Y
Residuo	%	$X \% Y$	Regresa el residuo de X entre Y
Raíz cuadrada	sqrt	sqrt(Y)	Raíz cuadrada de Y

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      int a = 20;
7      int b = 10;
8
9      int suma = a + b;
10     int resta = a - b;
11     int multiplicacion = a * b;
12     int division = a / b;
13
14     cout << "El resultado de la suma es: " << suma;
15     cout << "El resultado de la resta es: " << resta;
16     cout << "El resultado de la multiplicacion es: " << multiplicacion;
17     cout << "El resultado de la division es: " << division;
18
19     return 0;
20 }
```


Espacio de Preguntas

RESUMEN

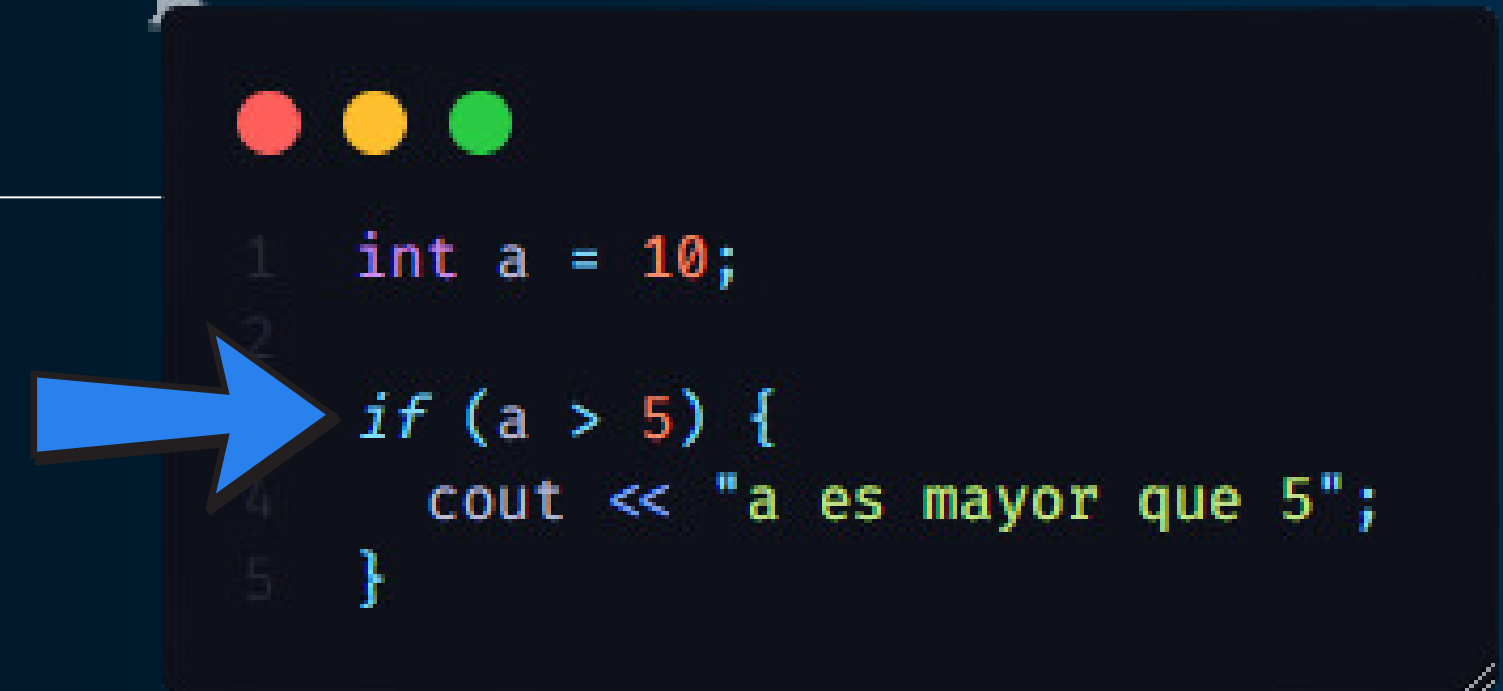
- ✓ La función **main** es el punto de inicio de cualquier programa en C++. Sin **main**, el programa no se ejecuta.
- ✓ Los tipos de datos definen qué tipo de información puede almacenar una variable (int, double, char, bool, string).
- ✓ La entrada y salida de datos en C++ se realiza con **cin** para leer valores del usuario y **cout** para mostrar información en pantalla.

Condicionales

Introducción a C++

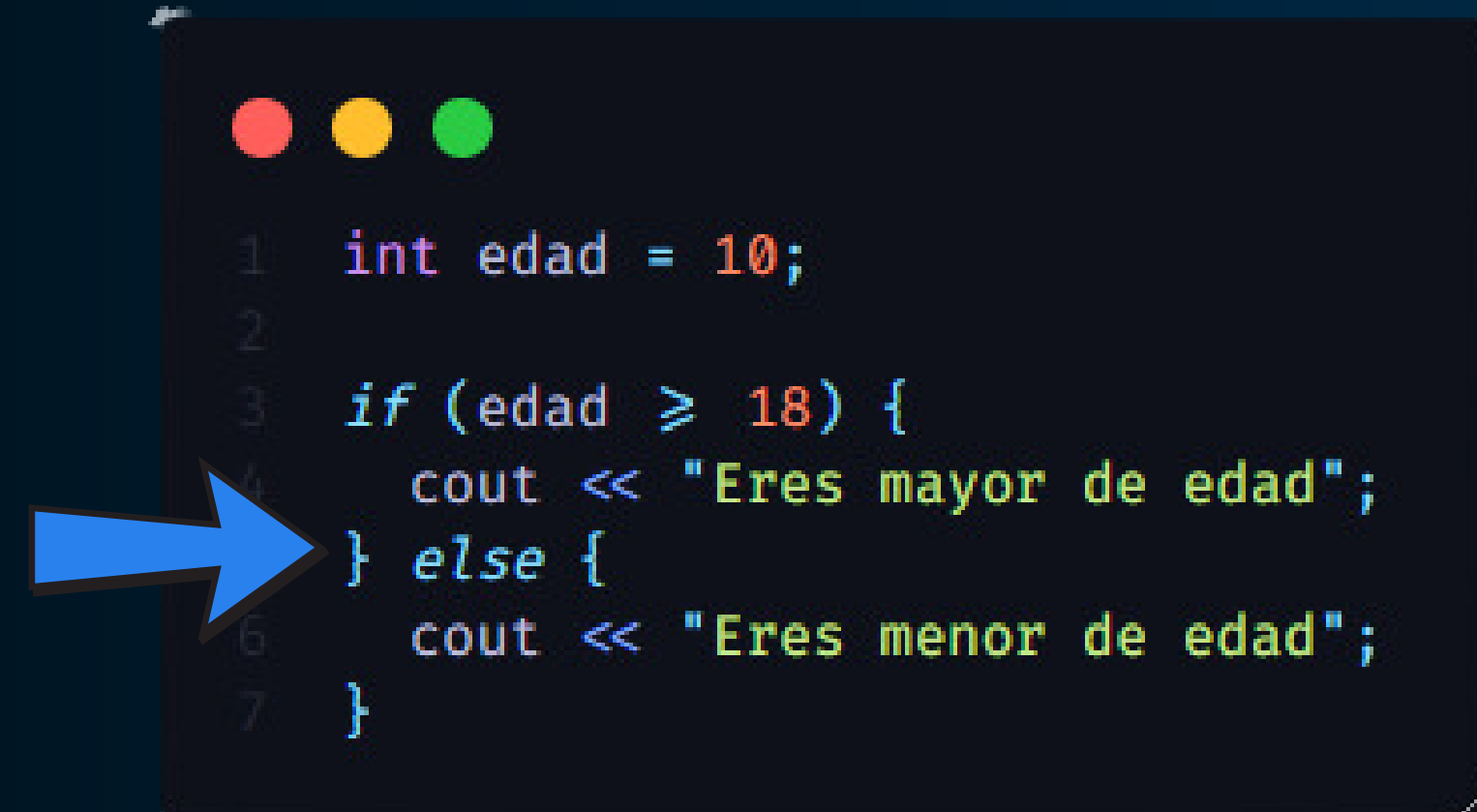
- Los condicionales nos permiten realizar una acción siempre y cuando se cumpla o no una condición.

Operador	Sintaxis	Significado
<	a < b	Menor que
<=	a <= b	Menor o igual que
>	a > b	Mayor que
>=	a >= b	Mayor o igual que
==	a == b	Igual a
!=	a != b	Distinto de
!	!a	Negación lógica
&&	a && b	AND lógico
	a b	OR lógico



```

1  int a = 10;
2
3  if (a > 5) {
4      cout << "a es mayor que 5";
5  }
  
```



```

1  int edad = 10;
2
3  if (edad >= 18) {
4      cout << "Eres mayor de edad";
5  } else {
6      cout << "Eres menor de edad";
7  }
  
```

else if – switch case

- Podemos definir múltiples condiciones para una sola variable.

```

1  int x = 10;
2
3  if (x < 10) {
4      cout << "x es menor que 10";
5  } else if (x == 10) {
6      cout << "x es igual a 10";
7  } else if (x > 10) {
8      cout << "x es mayor que 10";
9  }

```

```

1  int x = 10;
2
3  switch (x) {
4      case 10: {
5          cout << "x es igual a 10";
6          break;
7      }
8      case 20: {
9          cout << "x es igual a 20";
10         break;
11     }
12     case 30: {
13         cout << "x es igual a 30";
14         break;
15     }
16     default: {
17         cout << "x no es 10, 20 ni 30";
18     }
19 }

```

Operadores lógicos

- En ocasiones queremos saber **si se cumple más de una condición a la vez**. Para ello usaremos los operadores lógicos.

&&	a && b	AND lógico
	a b	OR lógico



```

1  int a = 5;
2  int b = 10;
3
4  if (a == 5 && b == 10) {
5      cout << "a es 5 y b es 10";
6  }

```




```

1  int a = 5;
2  int b = 16;
3
4  if (a == 5 || b == 10) {
5      cout << "Uno de los dos es verdadero o ambos lo son";
6  }

```



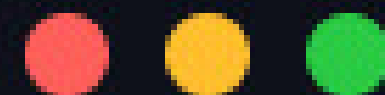
Bucles

Introducción a C++

- Los bucles permiten **repetir instrucciones** múltiples veces sin tener que escribirlas una por una.
- En C++, existen tres estructuras principales de repetición: **for**, **while** y **do-while**.



```
1  for (int i = 0; i < 10; i++) {
2      cout << i << endl;
3  }
```



```
1  int j = 0;
2
3  while (j < 10) {
4      cout << j << endl;
5      j++;
6  }
```

- Las variables "i" y "j" se consideran "**variables contadoras**" porque empiezan en 0 y cuentan uno por uno hasta que se cumpla una determinada condición.

- Las variables "suma" y "factorial" se consideran "**variables acumulativas**" porque cambian su valor original acumulado diferentes valores hasta que se cumple determinada condición.



```
1  int suma = 0;
2
3  while (suma < 50) {
4      suma += 10;
5  }
6  cout << "La suma es: " << suma;
```



```
1  int n = 5;
2  int factorial = 1;
3
4  while (n > 0) {
5      factorial *= n;
6      n--;
7  }
8  cout << "El factorial de 5 es: " << factorial;
```

Entrada de datos con Bucles

- Muchas veces tendremos la necesidad de recibir "N" valores que nos servirán para hallar un resultado final.

◆ Enunciado

Dado un conjunto de "n" números enteros, calcula su promedio.

◆ Entrada

Un entero "n" ($1 \leq n \leq 100000$), que indica la cantidad de números.
n enteros " x_i " ($-10^6 \leq x_i \leq 10^6$), los números que se deben promediar.

◆ Salida

Un número decimal representando el promedio de los "n" valores.

◆ Ejemplo de Entrada

```
4
5 10 15 20
```

◆ Ejemplo de Salida

```
12.5
```

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      int n; // Cantidad de números
7      cin >> n;
8
9      int suma = 0;
10     for (int i = 0; i < n; i++) {
11         int x;
12         cin >> x;
13         suma += x;
14     }
15
16     double promedio = (double) suma / n;
17     cout << "El promedio es: " << promedio << endl;
18
19     return 0;
20 }
```

GRACIAS

POR SU ATENCIÓN



CLUB DE
PROGRAMACIÓN
COMPETITIVA
UNFJSC